HIV Simulation Engine - Complete Model Guide

Version: 5.0 (with Validation Patches) **Purpose:** External validation and comprehensive feature documentation
Academic Context: Thesis research on HIV prevention interventions

Executive Summary

This is a discrete-time stochastic population model for HIV transmission dynamics and prevention interventions. The model is **inspired by STDSIM's conceptual framework** but reimplemented with modern computational best practices:

- **Label-based configuration** (not index-based)
- **Dynamic dimensions** (inferred from config)
- **Flexible time resolution** (days/months/years)
- **Comprehensive demography** (fertility, life tables, sexual debut)
- **Robust validation** (mass balance, conservation checks)
- **Time-scheduled interventions** (year-varying ART/PrEP)
- **Multi-scenario framework** (policy comparisons)
- **Advanced initialization** (census-based, relative weights)
- **Complete documentation**

Table of Contents

Part I: Core Model

1. Core Architecture
2. Time System
3. Population Representation
4. Group Definitions
5. Sexual Mixing
6. HIV Transmission
7. Disease Progression
8. Diagnosis & Treatment
9. PrEP Interventions
10. Demography (Basic)
11. Outputs

Part II: V4 Enhancements

12. Advanced Demography
13. Sexual Debut System

14. Childhood Pool
15. V4 Guardrails & Validation

Part III: V5 Features

16. Time-Scheduled Interventions
17. Multi-Scenario Framework
18. Policy Comparisons

Part IV: Advanced Customizations

19. Population Initialization Methods
20. Extended Age Groups & Open-Ended Bins
21. Validation Patches
22. Conservation Checks

Part V: Appendices

- Appendix A: Simulation Loop Order
- Appendix B: Mathematical Notation
- Appendix C: File Structure
- Appendix D: Configuration Hierarchy
- Appendix E: Performance
- Appendix F: Limitations
- Appendix G: Validation Checklist
- Appendix H: Version History

PART I: CORE MODEL

1. Core Architecture

1.1 Model Type

- **Population-based** (not individual/agent-based)
- **Discrete-time** (fixed timesteps, not continuous/event-driven)
- **Stochastic** (binomial/Poisson draws) with deterministic mode option

1.2 State Representation

Population stored as **counts in a multidimensional tensor**:

```
Shape: (n_age_groups, n_orientations, n_activities, n_hiv_stages,
        n_diagnosis, n_art, n_prep)
```

All dimensions **inferred dynamically from configuration** - no hard-coded limits.

1.3 Configuration System

- **Single unified JSON file** (config.json)
- **Label-based** (human-readable names, not numeric indices)
- **Self-documenting** (inline notes on every parameter)
- **Hierarchical structure** ordered by conceptual importance

1.4 Programming Language

- **Python 3.10+**
- **Dependencies:** NumPy, Pandas only
- **No cloud services** (local execution)

2. Time System

2.1 Flexible Time Units

Supports three time units:

- **Days** (365 per year)
- **Months** (12 per year)
- **Years** (1 per year)

2.2 Configurable Timestep

```
"time": {
  "unit": {"value": "months"},
  "step_size": {"value": 12},  // 12 months = yearly timestep
  "start_year": {"value": 1980},
  "end_year": {"value": 2020}
}
```

2.3 Rate Conversion

Annual rates automatically converted to per-timestep:

```
p_step = 1 - (1 - p_annual)^(years_per_step)
```

2.4 Number of Timesteps

Calculated as:

```
n_steps = (end_year - start_year) × (units_per_year / step_size)
```

Example:

- 1980-2020 (40 years)
- Monthly units, 12-month steps
- Result: 40 timesteps

3. Population Representation

3.1 State Tensor Dimensions

Dimension	Description	Examples	Configurable
Age	5-year bins (or custom)	15-19, 20-24, ..., 65+	✅ Yes
Orientation	Sexual orientation	Female, Hetero_Male, MSM	✅ Yes
Activity	Sexual activity level	Low, Medium, High	✅ Yes
HIV Status	Disease stage	Susceptible, Acute, Chronic, AIDS	⚠️ Fixed order
Diagnosis	HIV diagnosis status	Undiagnosed, Diagnosed	❌ Fixed (2 states)
ART	Treatment status	Not on ART, On ART	❌ Fixed (2 states)
PrEP	Prevention status	None, Oral_PrEP, Lenacapavir, ...	✅ Dynamic

3.2 Example State

```
A 25-year-old MSM with high sexual activity, chronic HIV,
diagnosed, on ART, not on PrEP:

population[2, 2, 2, 2, 1, 1, 0] = count
           │  │  │  │  │  │  └─ PrEP: None
           │  │  │  │  │  └─ ART: On ART
           │  │  │  │  └─ Diagnosis: Diagnosed
           │  │  │  └─ HIV: Chronic
           │  │  └─ Activity: High
           │  └─ Orientation: MSM
           └─ Age: 25-29
```

3.3 Population Initialization

Two methods available (see Section 19 for details):

Method 1: Direct Counts (legacy, simple)

```
  "initialization_method": {"value": "direct_counts"},
  "initial_distribution": {
    "Female": {
      "15-19": {
        "Low": 5000,
        "Medium": 3000,
        "High": 1000
      }
    }
  }
```

Method 2: Relative Weights (census-based, advanced)

```
  "initialization_method": {"value": "relative_weights"},
  "target_population": {"value": 12713815},
  "p_msm": {"value": 0.10},
  "age_sex_distribution": {
    "15-19": {"female": 0.086, "male": 0.089},
    "20-24": {"female": 0.081, "male": 0.084},
    ...
  }
```

3.4 Validation

- No negative counts
- All values finite
- Mass balance preserved across transitions
- Comprehensive conservation checks (V5)

4. Group Definitions

4.1 Age Groups

Standard closed bins:

```
  "age_groups": [
    {"name": "15-19", "min": 15, "max": 19},
    {"name": "20-24", "min": 20, "max": 24},
    {"name": "25-29", "min": 25, "max": 29},
    ...
    {"name": "45-49", "min": 45, "max": 49}
  ]
```

Extended with open-ended bins (V5+):

```
  "age_groups": [
    {"name": "15-19", "min": 15, "max": 19},
```

```
    ...
    {"name": "60-64", "min": 60, "max": 64},
    {"name": "65+", "min": 65, "max": 110}  // Open-ended bin
  ]
```

Features:

- Any number of groups
- Any age ranges
- Open-ended bins supported (e.g., "65+")
- Used for age-specific rates (fertility, mortality)

See Section 20 for details.

4.2 Sexual Orientations

Default: Female, Heterosexual_Male, MSM

Extensible (can add orientations)

Mixing rules:

- Female ↔ Heterosexual_Male (heterosexual mixing)
- MSM ↔ MSM (MSM mixing)

4.3 Sexual Activity Groups

Configurable with contact rates:

```
  "sexual_activity_groups": [
    {"name": "Low", "contact_rate": 3.0},
    {"name": "Medium", "contact_rate": 10.0},
    {"name": "High", "contact_rate": 50.0}
  ]
```

Features:

- Any number of groups (not limited to 3)
- Contact rate = annual number of sexual partners
- Used in transmission calculation

Orientation-Specific Contact Rates (V5+):

By default, contact rates are the same across all orientations. To model epidemiological reality where MSM have higher contact rates than heterosexual men, who have higher rates than women, use orientation multipliers:

```
  "orientation_contact_multipliers": {
    "value": {
      "Female": 0.7,            // 30% fewer contacts
      "Heterosexual_Male": 1.0, // Baseline
```

```
    "MSM": 2.5                // 150% more contacts
  },
  "notes": "Multipliers applied to base contact rates. Defaults to 1.0 if not specified."
}
```

Effective contact rates:

```
Female-Low:          3.0 × 0.7 = 2.1
Heterosexual_Male-Low: 3.0 × 1.0 = 3.0
MSM-Low:             3.0 × 2.5 = 7.5

Female-High:         50.0 × 0.7 = 35.0
Heterosexual_Male-High: 50.0 × 1.0 = 50.0
MSM-High:            50.0 × 2.5 = 125.0
```

✅ **Backward compatible:** If not specified, all multipliers default to 1.0 (current behavior)

See PATCH_ORIENTATION_CONTACT_MULTIPLIERS.md for details.

4.4 HIV Disease Stages

Fixed order (do not modify):

1. Susceptible (HIV-negative)
2. Acute (recent infection, high viral load)
3. Chronic (established infection)
4. AIDS (advanced disease)

4.5 PrEP States

Dynamically built from interventions:

- Always includes "None"
- Adds enabled interventions (e.g., Oral_PrEP, Lenacapavir)

5. Sexual Mixing

5.1 Group-Based Mixing

Uses **mixing matrices**, not explicit partner matching.

5.2 Mixing Matrices

Define proportion of contacts from activity group *i* to activity group *j*:

```
"heterosexual": {
  "Low": {
```

```
    "Low": 0.7,     // 70% within low activity
    "Medium": 0.25,
    "High": 0.05
  },
  "Medium": { ... },
  "High": { ... }
}
```

Requirements:

- Rows must sum to 1.0 (normalized probability distribution)
- Values in [0, 1]
- Validated on load

5.3 Contact Calculation

For each timestep:

```
total_contacts[orient, activity] =
    population[orient, activity] × contact_rate × years_per_step
```

5.4 Contact Allocation

Contacts allocated via mixing matrix:

```
contacts_allocated[from_orient, from_act, to_orient, to_act] =
    total_contacts[from_orient, from_act] × mixing_matrix[from_act, to_act]
```

5.5 Condom Usage

Activity-specific condom effectiveness modifier:

```
"condom_usage": {
  "Low": 0.5,     // 50% effective
  "Medium": 0.7,
  "High": 0.9
}
```

6. HIV Transmission

6.1 Transmission Probability

Per-contact probability calculated as:

```
P = β(stage) × (1 − PrEP_efficacy) × condom_modifier × ART_modifier × prevalence
```

Where:

- `β(stage)` = Base transmission probability by HIV stage
- `PrEP_efficacy` = Weighted by PrEP coverage in susceptible population
- `condom_modifier` = Activity-based condom effectiveness
- `ART_modifier` = Reduction from ART coverage in infected population
- `prevalence` = Proportion HIV+ in partner group

6.2 Stage-Specific Transmission

```
"transmission": {
  "beta_acute": 0.015,     // 1.5% per act (acute)
  "beta_chronic": 0.004,   // 0.4% per act (chronic)
  "beta_aids": 0.008       // 0.8% per act (AIDS)
}
```

6.3 ART Effect on Transmission

Treatment as Prevention (TasP / U=U):

```
"art_transmission_reduction": 0.96  // 96% reduction when on ART
```

6.4 New Infections

Drawn stochastically:

```
n_infected ~ Binomial(n_contacts × prevalence, P)
```

Capped at susceptible population.

6.5 Infection Assignment

New infections:

- Move from Susceptible → Acute
- Remain Undiagnosed
- Not on ART
- Lose PrEP status (if any)
- Distributed across age groups proportionally

6.6 Transmission Sanity Checks (V5 Validation)

Automatic verification at every timestep:

- Infections ≤ susceptible population (hard cap)
- Contact counts never negative
- Prevalence calculations validated
- Transmission probabilities in [0, 1]

See Section 21.1 for details.

7. Disease Progression

7.1 HIV Stages

Progression pathway:

```
Susceptible → Acute → Chronic → AIDS
```

7.2 Progression Rates

Annual probabilities:

```
"progression": {
  "acute_to_chronic": 0.5,    // 50% per year
  "chronic_to_aids": 0.05     // 5% per year
}
```

Converted to per-timestep automatically.

7.3 Implementation

Binomial draws:

```
progressions ~ Binomial(population[stage], progression_rate)
```

Move counts from current stage to next stage.

7.4 Both Progressions in One Timestep

Individuals can progress twice in one timestep:

- Acute → Chronic → AIDS

Example: 100 in Acute stage, 50% progression rate

- 50 move to Chronic
- Of those 50, ~2-3 may progress to AIDS (5% rate)
- Final: 50 Acute, 47 Chronic, 3 AIDS

8. Diagnosis & Treatment

8.1 Diagnosis Cascade

Undiagnosed → Diagnosed

Two modes available:

Simple (constant rate):

```
"diagnosis": {
  "annual_rate": 0.15  // 15% diagnosed per year
}
```

Time-scheduled (V5):

```
"diagnosis_schedule": {
  "interpolation": "stepwise",
  "schedule": {
    "1980": 0.05,
    "1990": 0.10,
    "2000": 0.15,
    "2010": 0.20
  }
}
```

Only HIV+ individuals can be diagnosed.

8.2 ART Initiation

Diagnosed, not on ART → On ART

Simple:

```
"treatment": {
  "art_initiation_rate": 0.8  // 80% start ART per year
}
```

Time-scheduled (V5):

```
"art_schedule": {
  "interpolation": "stepwise",
```

```
    "initiation": {
      "schedule": {
        "1996": 0.1,
        "2000": 0.3,
        "2010": 0.7,
        "2020": 0.85
      }
    }
  }
```

8.3 ART Dropout

On ART → Not on ART

```
  "art_dropout_rate": 0.05  // 5% discontinue per year
```

Or time-scheduled:

```
  "art_schedule": {
    "discontinuation": {
      "schedule": {
        "1996": 0.15,
        "2010": 0.05
      }
    }
  }
```

8.4 ART Effects

Transmission:

```
  transmission_modifier = 1 - (proportion_on_ART × 0.96)
```

Mortality:

```
  hiv_mortality_on_ART = hiv_mortality × (1 - 0.90)  // 90% reduction
```

8.5 ART Mass Balance (V5 Validation)

Automatic verification:

- Diagnosed population tracked
- ART transitions balanced
- Conservation laws enforced
- No leakage in cascade

See Section 21.3 for details.

9. PrEP Interventions

9.1 Supported Interventions

- **Oral PrEP** (daily pills)
- **Lenacapavir** (injectable, 6-month duration)
- **Extensible** (can add new interventions)

9.2 PrEP Configuration

Simple (constant rates):

```
"Oral_PrEP": {
  "enabled": true,
  "dosing_interval": {"value": 3, "unit": "months"},
  "efficacy": 0.86,           // 86% effective
  "uptake_rate": 0.05,        // 5% start per year
  "dropout_rate": 0.2,        // 20% stop per year
  "uptake_modifier_by_activity": {
    "Low": 0.5,
    "Medium": 1.0,
    "High": 1.5
  }
}
```

Time-scheduled (V5):

```
"Oral_PrEP": {
  "enabled": true,
  "efficacy": 0.86,
  "prep_schedule": {
    "interpolation": "linear",
    "uptake": {
      "schedule": {
        "2012": 0.00,
        "2015": 0.02,
        "2020": 0.10
      }
    },
    "discontinuation": {
      "schedule": {
        "2012": 0.30,
        "2020": 0.15
      }
    }
  }
}
```

9.3 PrEP Transitions

Susceptible only can use PrEP:

- None → PrEP (uptake)
- PrEP → None (dropout)

HIV infection removes PrEP:

- Acute infections always have PrEP = None

9.4 PrEP Effect on Transmission

Reduces susceptibility:

```
prep_modifier = 1 − (proportion_on_prep × efficacy)
```

Applied in transmission probability calculation.

9.5 Dosing Intervals

Engine aligns dosing with timestep:

- 3-month oral prescription with 12-month timestep = annual
- 6-month injectable with 3-month timestep = every 2 steps

10. Demography (Basic)

10.1 Demography Modes

Three modes supported:

Closed:

- No births
- Aging and mortality only
- Population declines over time

Fixed Net Addition:

- Constant annual entrants
- Added to entry age group
- Simple population maintenance

Fertility Table:

- ASFR-based births (age-specific fertility rates)
- Year-specific multipliers

- Most realistic demography

10.2 Mortality

HIV Mortality:

```
"mortality": {
  "acute": 0.01,      // 1% per year
  "chronic": 0.02,    // 2% per year
  "aids": 0.25        // 25% per year
}
```

Background Mortality:

```
"demography": {
  "background_mortality": 0.008  // 0.8% per year
}
```

Or use life tables (see V4 section)

10.3 Aging

Every 5 years:

- Population moves up one age group
- Oldest group remains (or dies via mortality)

New entrants:

- Added to youngest group
- Scaled by timestep (annual_entrants × years_per_step)
- Distributed across orientations and activities

11. Outputs

11.1 File Outputs

Per-Run Files:

```
outputs/baseline_run_001.csv
outputs/baseline_run_002.csv
...
```

Aggregated Files:

```
outputs/baseline_aggregated.csv
```

Contains mean + quantiles (Q05, Q50, Q95) across runs.

11.2 Output Formats

- **CSV** (default)
- **Excel** (.xlsx) - optional

11.3 Timeseries Columns

Column	Description
step	Timestep number (0-indexed)
year	Calendar year (floating point)
new_infections	HIV infections this timestep
prevalence	HIV+ / total population
incidence	New infections / susceptible
hiv_deaths	HIV-related deaths
background_deaths	Non-HIV deaths
total_population	Total population
births	Births this timestep (if fertility enabled)
prep_coverage_*	PrEP coverage by intervention

11.4 Terminal Summaries

Single Run:

```
Year       Population HIV Prev %    New Inf   HIV Deaths    BG Deaths
1980        1,120,494       0.10          0            0        9,006
1985        1,076,649       0.10          0            0        8,756
...

Total new infections: 0
Final prevalence: 0.10%
PrEP Coverage: Oral_PrEP: 0.00%, Lenacapavir: 0.00%
```

Multi-Run:

```
Final Year (2020) Statistics:
Metric                      Mean          Q05          Q50          Q95
```

```
Total Population          819,320      819,114      819,170      819,631
HIV Prevalence (%)           0.10         0.10         0.10         0.10
```

11.5 State Counts (Optional)

Full population tensor at final timestep.

Size: `n_age × n_orient × n_activity × n_hiv × n_dx × n_art × n_prep`

Large files - disabled by default.

PART II: V4 ENHANCEMENTS

12. Advanced Demography (V4)

12.1 Life Tables

Optional replacement for constant background mortality.

Structure:

```
"life_table": {
  "enabled": {"value": true},
  "survival_by_age_limit": {
    "value": [
      {"age_limit": 1, "male_survival": 0.99622, "female_survival": 0.99687},
      {"age_limit": 5, "male_survival": 0.99913, "female_survival": 0.99943},
      {"age_limit": 10, "male_survival": 0.99965, "female_survival": 0.99975},
      ...
      {"age_limit": 110, "male_survival": 0.00001, "female_survival": 0.00005}
    ]
  }
}
```

Conversion to Mortality:

```
survival_per_step = survival_annual ^ years_per_step
mortality_per_step = 1 - survival_per_step
```

Application:

- Sex-specific (male vs female)
- Age-specific (lookup by age midpoint)
- Applied to all HIV stages as background mortality

12.2 HIV Mortality Integration

Two modes:

Excess (default):

```
total_mortality = background_mortality + hiv_mortality
```

Override:

```
total_mortality = hiv_mortality (background ignored for HIV+)
```

HIV mortality reduced by 90% when on ART.

12.3 Fertility Implementation

Age-Specific Fertility Rates (ASFR):

```
"fertility": {
  "enabled": {"value": true},
  "asfr_by_age_limit": {
    "value": [
      {"age_limit": 20, "birth_rate": 0.0028},
      {"age_limit": 25, "birth_rate": 0.0850},
      {"age_limit": 30, "birth_rate": 0.1150},
      {"age_limit": 35, "birth_rate": 0.1050},
      {"age_limit": 40, "birth_rate": 0.0450},
      {"age_limit": 45, "birth_rate": 0.0050},
      {"age_limit": 50, "birth_rate": 0.0001}
    ]
  }
}
```

Birth Calculation:

```
births = women_count × asfr × years_per_step × year_multiplier
```

Year Multipliers:

```
"year_multipliers": {
  "values": {
    "1980": 1.0,
    "1990": 0.98,
    "2000": 0.95,
    "2010": 0.92,
    "2020": 0.90
  }
```

```
}
```

Interpolated between years.

Newborn Assignment:

- Sex ratio at birth (default 51.2% male)
- Entry age group (usually 15-19 for sexual debut)
- Activity distribution (usually all "Low")
- Susceptible, undiagnosed, no ART, no PrEP

12.4 Population Adjustments

Optional exogenous scaling for calibration:

```
"population_adjustments": {
  "enabled": true,
  "apply_to": "total",  // or by_age, by_orientation
  "year_multipliers": {
    "1980": 1.0,
    "1990": 1.02,    // 2% increase
    "2000": 1.05     // 5% increase
  }
}
```

Applied at start of each timestep before transmission.

13. Sexual Debut System (V4)

13.1 Overview

Sexual debut is modeled as a **continuous per-timestep transition** applied to individuals in debut-eligible childhood age groups, ensuring smooth inflow into the sexually active population across all timesteps.

13.2 Key Features

Continuous hazard approach:

- No discrete cohort spikes
- Per-timestep transition probabilities
- Smooth debut throughout age window

Flush-on-aging:

- Individuals remaining in debut-eligible bin at age-out are transitioned to sexually active population
- Prevents artificial exponential tail
- Ensures near-complete debut within age window

Timestep robustness:

- Works with any timestep resolution (daily, monthly, yearly)
- Consistent behavior across different configurations

13.3 Configuration

```
"sexual_debut": {
  "enabled": {"value": true},
  "debut_age_group": {"value": "10-14"},
  "target_age_group": {"value": "15-19"},
  "annual_transition_rate": {"value": 0.20}
}
```

Parameters:

- `debut_age_group` : Which childhood bin contains debut-eligible individuals
- `target_age_group` : Where they enter sexually active population
- `annual_transition_rate` : Annual probability of debut (converted to per-timestep)

13.4 Implementation Details

Per-timestep transition:

```
p_debut_step = 1 - (1 - 0.20)^(years_per_step)
n_debuting ~ Binomial(debut_eligible_count, p_debut_step)
```

Orientation assignment at debut:

- Males: Split into MSM (p_msm) and Heterosexual_Male (1 - p_msm)
- Females: All assigned to Female orientation
- Activity: Typically all "Low" (configurable)

Flush-on-aging:

- When debut_age_group ages out → all remaining individuals transition
- Prevents accumulation in childhood bins
- Ensures complete sexual debut before aging into adult bins

13.5 Benefits

✅ **No discrete spikes** - smooth inflow every timestep ✅ **Timestep-robust** - works with any time resolution ✅ **Near-complete debut** - flush ensures everyone debuts by age-out ✅ **Realistic dynamics** - continuous hazard mimics real sexual debut patterns

14. Childhood Pool (V4)

14.1 Overview

The childhood pool tracks individuals aged 0-14 (pre-sexual debut) separately from the sexually active population tensor.

Purpose:

- Model complete demographic structure (births → childhood → sexual debut → aging → death)
- Support fertility-based demography
- Enable sexual debut modeling

14.2 Structure

Childhood pool dimensions:

```
Shape: (n_childhood_age_groups, n_sexes)
```

Example:

```
Childhood bins: ["0–4", "5–9", "10–14"]
Sexes: ["female", "male"]

childhood[0, 0] = females aged 0–4
childhood[1, 1] = males aged 5–9
childhood[2, 0] = females aged 10–14 (debut-eligible)
```

14.3 Configuration Options

Option 1: Auto (calculate from adult population)

```
"childhood_pool": {"value": "auto"}
```

- Infers childhood counts from age_sex_distribution
- Extracts ages <15 from distribution
- Good for census-based initialization

Option 2: Disabled (no childhood modeling)

```
"childhood_pool": {"value": "disabled"}
```

- No childhood pool
- All births enter sexually active population directly at entry age (e.g., 15-19)
- Suitable for adult-only models

Option 3: Manual counts

```
  "childhood_pool": {
    "value": {
      "0-4": {"female": 50000, "male": 52000},
      "5-9": {"female": 48000, "male": 50000},
      "10-14": {"female": 46000, "male": 48000}
    }
  }
```

14.4 Childhood Transitions

Inflows:

- Births (from fertility module)
- Immigration (if enabled)

Outflows:

- Sexual debut (10-14 → 15-19)
- Childhood mortality (life table or constant rate)
- Aging (0-4 → 5-9 → 10-14)

Not modeled in childhood:

- HIV status (all susceptible)
- Sexual activity (none)
- Interventions (no PrEP/ART)

14.5 Integration with Sexual Debut

When sexual debut is enabled:

1. Childhood pool accumulates births
2. Individuals age through childhood bins (0-4 → 5-9 → 10-14)
3. Debut-eligible bin (10-14) transitions to sexually active population
4. Orientation and activity assigned at debut
5. Flush-on-aging ensures complete transition

15. V4 Guardrails & Validation

15.1 Flush-on-Aging Guardrails

Purpose: Prevent accumulation of individuals in age bins due to incomplete transitions.

Implementation:

- When an age group ages out, residual population is flushed to next bin
- Applies to both childhood aging and sexually active aging

- Ensures no "stuck" individuals in age groups

Example:

```
10–14 childhood bin aging to 15–19:
– Normal debut transitions: 95% transitioned during age window
– Flush-on-aging: Remaining 5% forced to transition at age-out
– Result: 100% transition, no accumulation
```

15.2 Conservation Checks

Per-timestep validation:

- Total population before = total population after (accounting for births/deaths)
- No negative counts in any dimension
- All values finite (no NaN/Inf)
- Orientation counts sum correctly
- Activity counts sum correctly

Raises ValueError if violated.

15.3 Configuration Validation

Enhanced validation on config load:

- Childhood age groups consistent with sexual debut configuration
- Entry age group exists in sexually active age groups
- Fertility age groups covered by age_groups definition
- Life table extends to maximum age
- No orphaned or undefined group references

PART III: V5 FEATURES

16. Time-Scheduled Interventions (V5)

16.1 Overview

V5 introduces **time-varying intervention rates** that change by calendar year, enabling realistic modeling of historical scale-up and future projections.

Applies to:

- ART initiation rates
- ART discontinuation rates

- Diagnosis rates
- PrEP uptake rates
- PrEP dropout rates

16.2 ART Schedules

Configuration:

```
"art_schedule": {
  "enabled": {"value": true},
  "interpolation": {"value": "stepwise"},
  "initiation": {
    "schedule": {
      "value": {
        "1996": 0.10,    // 10% initiation in 1996
        "2000": 0.30,    // 30% by 2000
        "2010": 0.70,    // 70% by 2010
        "2020": 0.85     // 85% by 2020
      }
    }
  },
  "discontinuation": {
    "schedule": {
      "value": {
        "1996": 0.15,    // 15% dropout in early ART era
        "2010": 0.05     // 5% dropout in modern era
      }
    }
  }
}
```

Interpolation modes:

- `"stepwise"` : Step function (rate constant until next year)
- `"linear"` : Linear interpolation between years

16.3 Diagnosis Schedules

```
"diagnosis_schedule": {
  "enabled": {"value": true},
  "interpolation": {"value": "linear"},
  "schedule": {
    "value": {
      "1980": 0.05,
      "1990": 0.10,
      "2000": 0.15,
      "2010": 0.20,
      "2020": 0.25
    }
  }
}
```

16.4 PrEP Schedules

Per-intervention schedules:

```
"interventions": {
  "Oral_PrEP": {
    "enabled": true,
    "efficacy": 0.86,
    "prep_schedule": {
      "enabled": {"value": true},
      "interpolation": {"value": "linear"},
      "uptake": {
        "schedule": {
          "value": {
            "2012": 0.00,    // Pre-approval
            "2015": 0.02,    // Early adoption
            "2020": 0.10     // Scale-up
          }
        }
      },
      "discontinuation": {
        "schedule": {
          "value": {
            "2012": 0.30,    // High dropout initially
            "2020": 0.15     // Improved retention
          }
        }
      }
    }
  }
}
```

16.5 Implementation Details

Rate lookup:

- At each timestep, current year is calculated
- Schedule is queried for rate at that year
- Interpolation applied between defined years
- Extrapolation uses nearest boundary value

Example (linear interpolation):

```
Years defined: 2000 (rate=0.3), 2010 (rate=0.7)
Query year 2005:
  rate = 0.3 + (2005-2000)/(2010-2000) × (0.7-0.3) = 0.5
```

16.6 Backward Compatibility

If schedules are not enabled, engine uses constant rates:

```
"treatment": {
  "art_initiation_rate": 0.8  // Constant across all years
```

```
}
```

Both modes work simultaneously - use schedules for calibration, constants for exploration.

17. Multi-Scenario Framework (V5)

17.1 Overview

V5 introduces a **multi-scenario runner** that executes multiple policy scenarios with config overrides in a single run.

Use cases:

- Parameter sensitivity analysis
- Intervention comparisons (status quo vs. scale-up)
- Policy counterfactuals (with vs. without PrEP)
- Time-varying coverage scenarios

17.2 Configuration

Enable scenarios in base config:

```
"scenarios": {
  "enabled": {"value": true},
  "scenario_list": {
    "value": [
      {
        "name": "baseline",
        "description": "Status quo with current ART/PrEP coverage",
        "overrides": {}
      },
      {
        "name": "no_prep",
        "description": "Counterfactual: No PrEP interventions",
        "overrides": {
          "interventions.Oral_PrEP.enabled": false,
          "interventions.Lenacapavir.enabled": false
        }
      },
      {
        "name": "prep_scaleup",
        "description": "Aggressive PrEP scale-up scenario",
        "overrides": {
          "interventions.Oral_PrEP.prep_schedule.uptake.schedule.value.2025": 0.25,
          "interventions.Lenacapavir.prep_schedule.uptake.schedule.value.2025": 0.15
        }
      }
    ]
  }
}
```

17.3 Override Syntax

Dot-notation path:

```
"path.to.parameter": new_value
```

Examples:

```
"overrides": {
  "interventions.Oral_PrEP.efficacy": 0.95,
  "treatment.art_initiation_rate": 0.90,
  "time.end_year": 2030
}
```

Nested updates:

```
"overrides": {
  "art_schedule.initiation.schedule.value.2025": 0.95
}
```

17.4 Running Scenarios

Command:

```
python run_scenarios.py config_v5.json
```

Output:

```
=== Running Multi-Scenario Analysis ===
Base config: config_v5.json

Scenario 1/3: baseline
Description: Status quo with current ART/PrEP coverage
Overrides: None
[Running 10 seeds...]
✓ Scenario baseline completed

Scenario 2/3: no_prep
Description: Counterfactual: No PrEP interventions
Overrides:
  interventions.Oral_PrEP.enabled = False
  interventions.Lenacapavir.enabled = False
[Running 10 seeds...]
✓ Scenario no_prep completed

Scenario 3/3: prep_scaleup
Description: Aggressive PrEP scale-up scenario
Overrides:
  interventions.Oral_PrEP.prep_schedule.uptake.schedule.value.2025 = 0.25
```

```
  interventions.Lenacapavir.prep_schedule.uptake.schedule.value.2025 = 0.15
[Running 10 seeds...]
✓ Scenario prep_scaleup completed

=== All scenarios completed ===
```

17.5 Scenario Outputs

Per-scenario directories:

```
outputs/
├── baseline/
│   ├── baseline_run_001.csv
│   ├── baseline_run_002.csv
│   └── baseline_aggregated.csv
├── no_prep/
│   ├── no_prep_run_001.csv
│   └── no_prep_aggregated.csv
└── prep_scaleup/
    ├── prep_scaleup_run_001.csv
    └── prep_scaleup_aggregated.csv
```

Comparison ready:

- Each scenario has identical structure
- Easy to compare aggregated.csv files
- Scenario name embedded in filenames

17.6 Parallel vs Sequential

Default: Sequential (scenarios run one after another)

- Easier debugging
- Lower memory usage
- Clear progress tracking

Parallel: Not yet implemented

- Future enhancement
- Would speed up multi-scenario runs

18. Policy Comparisons (V5)

18.1 Baseline Freeze & Reuse

Problem: When comparing multiple intervention scenarios, you want a common baseline to measure impact against.

Solution: V5 supports baseline freezing and reuse.

Configuration:

```
"policy": {
  "enabled": {"value": true},
  "baseline_freeze_year": {"value": 2020},
  "reuse_baseline": {"value": true}
}
```

How it works:

1. **Baseline run:** Simulation from start_year to baseline_freeze_year (e.g., 1980-2020)
2. **Freeze state:** Population state at 2020 is saved
3. **Policy scenarios:** Multiple scenarios start from frozen 2020 state and run 2020-2030
4. **Comparison:** All scenarios share identical 1980-2020 history

18.2 Benefits

✅ **Common baseline:** All scenarios start from same 2020 population state ✅ **Faster runs:** Don't re-run 1980-2020 for each scenario ✅ **Fair comparison:** Isolates impact of policy changes ✅ **Reduced variance:** Shared history reduces noise in comparisons

18.3 Example Use Case

Research question: What is the impact of PrEP scale-up from 2020-2030?

Approach:

1. Run baseline 1980-2020 (historical calibration)
2. Freeze at 2020
3. Run scenarios 2020-2030:
   - Status quo (current coverage)
   - Modest scale-up
   - Aggressive scale-up
4. Compare infections averted between scenarios

Config:

```
"time": {
  "start_year": 1980,
  "end_year": 2030
},
"policy": {
  "enabled": true,
  "baseline_freeze_year": 2020,
  "reuse_baseline": true
},
"scenarios": {
  "enabled": true,
  "scenario_list": [
    {
      "name": "status_quo",
```

```
      "overrides": {}
    },
    {
      "name": "modest_scaleup",
      "overrides": {
        "interventions.Oral_PrEP.prep_schedule.uptake.schedule.value.2025": 0.15
      }
    },
    {
      "name": "aggressive_scaleup",
      "overrides": {
        "interventions.Oral_PrEP.prep_schedule.uptake.schedule.value.2025": 0.30
      }
    }
  ]
}
```

18.4 Validation During Policy Runs

All validation patches (transmission sanity, mass balance, ART balance) remain active during policy scenarios, ensuring consistency and correctness.

PART IV: ADVANCED CUSTOMIZATIONS

19. Population Initialization Methods

19.1 Overview

V5 supports **two population initialization methods**:

1. **Direct Counts** (legacy): Specify exact counts for each orientation × age × activity combination
2. **Relative Weights** (census-based): Provide age/sex distributions and let engine allocate

19.2 Method 1: Direct Counts

When to use:

- Small, stylized populations
- Full control over every cell
- Legacy configurations

Configuration:

```
"population": {
  "initialization_method": {"value": "direct_counts"},
  "initial_distribution": {
    "value": {
```

```
      "Female": {
        "15-19": {"Low": 5000, "Medium": 3000, "High": 1000},
        "20-24": {"Low": 4500, "Medium": 2800, "High": 900},
        ...
      },
      "Heterosexual_Male": {
        "15-19": {"Low": 4500, "Medium": 2700, "High": 800},
        ...
      },
      "MSM": {
        "15-19": {"Low": 450, "Medium": 270, "High": 80},
        ...
      }
    }
  }
}
```

Characteristics:

- Explicit counts for every cell
- Manual MSM allocation
- Manual activity distribution
- Total population = sum of all cells

19.3 Method 2: Relative Weights (Census-Based)

When to use:

- Census-based initialization
- Large populations (millions)
- Automatic MSM allocation
- Simplify configuration

Configuration:

```
"population": {
  "initialization_method": {"value": "relative_weights"},
  "target_population": {"value": 12713815},
  "p_msm": {"value": 0.10},
  "activity_distribution": {
    "value": {
      "Low": 0.60,
      "Medium": 0.30,
      "High": 0.10
    }
  },
  "age_sex_distribution": {
    "value": {
      "0-4": {"female": 0.0950, "male": 0.0985},
      "5-9": {"female": 0.0920, "male": 0.0955},
      "10-14": {"female": 0.0890, "male": 0.0925},
      "15-19": {"female": 0.0860, "male": 0.0895},
      "20-24": {"female": 0.0810, "male": 0.0840},
      ...
      "60-64": {"female": 0.0450, "male": 0.0420},
```

```
      "65+": {"female": 0.0890, "male": 0.0750}
    }
  },
  "childhood_pool": {"value": "auto"},
  "hiv": {
    "initial_prevalence": {"value": 0.001}
  }
}
```

Characteristics:

- Age/sex weights (proportions that sum to 1.0)
- Target population size
- Global MSM proportion (p_msm)
- Automatic allocation to orientations and activities
- Sexual debut boundary enforcement

19.4 Relative Weights Algorithm

Step 1: Normalize age × sex distributions

```
total_weight = Σ(age_sex_distribution values)
normalized[age, sex] = raw[age, sex] / total_weight
```

Step 2: Apply sexual debut boundary

```
For each age group:
  if age < 15:
    → Goes to childhood pool
  else:
    → Eligible for sexual activity
```

Step 3: Split males into orientations

```
For age ≥ 15:
  males_total = target_population × normalized_weight[age, male]
  msm_count = males_total × p_msm
  hetero_male_count = males_total × (1 − p_msm)
  female_count = target_population × normalized_weight[age, female]
```

Step 4: Assign activity levels

```
For each orientation and age:
  Low = count × activity_distribution[Low]
  Medium = count × activity_distribution[Medium]
  High = count × activity_distribution[High]
```

Step 5: Populate childhood pool

```
For age < 15:
  childhood[age_bin, female] = target_population × normalized_weight[age, female]
  childhood[age_bin, male] = target_population × normalized_weight[age, male]
```

Step 6: Seed initial HIV

```
For each cell:
  susceptible = count × (1 − initial_prevalence)
  acute = count × initial_prevalence
```

19.5 Conservation Checks (Mandatory)

Six checks run automatically:

1. **Total population:** Allocated population = target population (±0.001%)
2. **Sexually active:** Sum of sexually active = expected (ages ≥15)
3. **Childhood pool:** Sum of childhood = expected (ages <15)
4. **Female orientation:** Sum of Female = expected
5. **Heterosexual_Male orientation:** Sum of Hetero_Male = expected (90% of males if p_msm=0.10)
6. **MSM orientation:** Sum of MSM = expected (10% of males if p_msm=0.10)
7. **Activity distribution:** Each activity sums correctly
8. **No negatives:** All counts ≥ 0

Fail-fast: If any check fails, initialization aborts with detailed error message.

19.6 Example: Census-Based Initialization

Input (real census data):

```
Total population: 12,713,815
Age groups: 15–19 through 65+
Sex distribution: ~50% male, ~50% female
Desired MSM proportion: 10% of males
Activity: 60% Low, 30% Medium, 10% High
```

Result:

```
✓ Total population: 12,713,815
✓ Sexually active (≥15): ~11,500,000
✓ Childhood (<15): ~1,213,815
✓ Females: ~6,356,908 (50%)
✓ Heterosexual_Male: ~5,721,217 (45%, 90% of males)
✓ MSM: ~635,691 (5%, 10% of males)
✓ Activities allocated correctly
```

19.7 Choosing a Method

Criteria	Direct Counts	Relative Weights
Population size	Small (<100k)	Large (millions)
Data source	Manual design	Census data
MSM allocation	Manual	Automatic
Configuration effort	High (every cell)	Low (age/sex only)
Flexibility	Full control	Standardized
Validation	Basic	Comprehensive
Use case	Stylized models	Calibration

20. Extended Age Groups & Open-Ended Bins

20.1 Overview

V5 supports **extended age groups** beyond the typical 15-49 range, including **open-ended bins** like "65+", "70+", etc.

Use cases:

- Census-compatible age structures
- Modeling older populations
- Complete demographic lifecycle

20.2 Standard vs Extended Age Groups

Standard (V3/V4):

```
"age_groups": [
  {"name": "15-19", "min": 15, "max": 19},
  {"name": "20-24", "min": 20, "max": 24},
  {"name": "25-29", "min": 25, "max": 29},
  {"name": "30-34", "min": 30, "max": 34},
  {"name": "35-39", "min": 35, "max": 39},
  {"name": "40-44", "min": 40, "max": 44},
  {"name": "45-49", "min": 45, "max": 49}
]
```

Extended (V5+):

```
"age_groups": [
  {"name": "15-19", "min": 15, "max": 19},
  {"name": "20-24", "min": 20, "max": 24},
  {"name": "25-29", "min": 25, "max": 29},
  {"name": "30-34", "min": 30, "max": 34},
```

```
  {"name": "35-39", "min": 35, "max": 39},
  {"name": "40-44", "min": 40, "max": 44},
  {"name": "45-49", "min": 45, "max": 49},
  {"name": "50-54", "min": 50, "max": 54},
  {"name": "55-59", "min": 55, "max": 59},
  {"name": "60-64", "min": 60, "max": 64},
  {"name": "65+", "min": 65, "max": 110}  // Open-ended bin
]
```

20.3 Open-Ended Bin Support

Syntax:

- `"65+"` - ages 65 and above
- `"70+"` - ages 70 and above
- `"75+"` - ages 75 and above

Configuration:

```
{"name": "65+", "min": 65, "max": 110}
```

How it works:

- The `"+"` character is parsed during initialization
- Minimum age extracted: `"65+"` → min_age = 65
- Used for sexual debut boundary check (<15 vs ≥15)
- Functions as oldest age accumulator during aging

20.4 Age Group Parsing

Three formats supported:

1. **Closed bins:** `"15-19"` → min_age = 15, max_age = 19
2. **Open-ended bins:** `"65+"` → min_age = 65
3. **Single values:** `"15"` → min_age = 15

Parsing logic:

```
if '-' in age_group:
    min_age = int(age_group.split('-')[0])
elif '+' in age_group:
    min_age = int(age_group.replace('+', ''))
else:
    min_age = int(age_group)
```

20.5 Census Alignment

Problem: Census data often has 20+ age groups (0-4, 5-9, ..., 105-109) but model may only need 15-65+.

Solution: Use relative weights initialization with extended age groups.

Example:

```
"age_sex_distribution": {
  "value": {
    "0-4": {"female": 0.095, "male": 0.099},
    "5-9": {"female": 0.092, "male": 0.096},
    "10-14": {"female": 0.089, "male": 0.093},
    "15-19": {"female": 0.086, "male": 0.089},
    ...
    "60-64": {"female": 0.045, "male": 0.042},
    "65+": {"female": 0.089, "male": 0.075}  // Aggregated 65-109
  }
}
```

Aggregation:

- If census has "65-69", "70-74", ..., "105-109"
- Manually aggregate into single "65+" bin
- Sum the proportions: `"65+" = sum(65-69, 70-74, ..., 105-109)`

20.6 Life Table Extension

When using extended age groups, **life tables must extend** to cover all ages:

```
"life_table": {
  "survival_by_age_limit": {
    "value": [
      {"age_limit": 1, "male_survival": 0.99622, "female_survival": 0.99687},
      ...
      {"age_limit": 65, "male_survival": 0.98234, "female_survival": 0.98876},
      {"age_limit": 70, "male_survival": 0.96543, "female_survival": 0.97654},
      {"age_limit": 75, "male_survival": 0.93210, "female_survival": 0.95432},
      {"age_limit": 80, "male_survival": 0.87654, "female_survival": 0.91234},
      {"age_limit": 110, "male_survival": 0.00001, "female_survival": 0.00005}
    ]
  }
}
```

20.7 Fertility Extension

If modeling fertility in older age groups:

```
"fertility": {
  "asfr_by_age_limit": {
    "value": [
      {"age_limit": 20, "birth_rate": 0.0028},
      ...
      {"age_limit": 50, "birth_rate": 0.0001},
      {"age_limit": 110, "birth_rate": 0.0000}  // No fertility 50+
    ]
```

```
    }
  }
```

20.8 Benefits

✅ **Census-compatible:** Match real demographic data structures ✅ **Complete lifecycle:** Model full age range 0-110 ✅ **Older populations:** Important for aging HIV+ populations ✅ **Flexible:** Any age structure supported ✅ **Backward compatible:** Standard 15-49 still works

21. Validation Patches

21.1 Transmission Sanity Checks

Purpose: Ensure transmission calculations are physically plausible at every timestep.

Checks:

1. **Infections ≤ susceptible population**

   ```
   new_infections[orientation, age, activity] ≤ susceptible[orientation, age, activity]
   ```

   Hard cap enforced - cannot infect more people than exist.
2. **Contact counts non-negative**

   ```
   total_contacts[orientation, activity] ≥ 0
   ```

3. **Prevalence in [0, 1]**

   ```
   0 ≤ prevalence[partner_group] ≤ 1
   ```

4. **Transmission probabilities in [0, 1]**

   ```
   0 ≤ transmission_probability ≤ 1
   ```

Implementation:

- Runs every timestep during transmission module
- ValueError raised if any check fails
- Provides detailed diagnostic information

Example error:

```
ERROR: Transmission sanity check failed at timestep 15 (year 1995)
  New infections (1500) exceed susceptible population (1200) in:
    Orientation: MSM, Age: 25-29, Activity: High
```

21.2 Global Population Mass Balance

Purpose: Ensure population accounting is correct - no "leakage" or spontaneous generation.

Equation:

```
population[t+1] = population[t] + births[t] - deaths[t] + entrants[t]
```

Checked every timestep:

```
expected_pop = initial_pop + births - deaths + entrants
actual_pop = current_pop
assert abs(expected_pop - actual_pop) < tolerance
```

Tracks:

- Total population before step
- Births added
- HIV deaths
- Background deaths
- New entrants (if applicable)
- Total population after step

Tolerance: ±1 person (accounts for rounding)

Example error:

```
ERROR: Mass balance violation at timestep 20 (year 2000)
  Expected population: 1,150,000
  Actual population: 1,149,985
  Discrepancy: -15 (0.001%)

  Accounting:
    Initial: 1,148,500
    Births: +2,500
    HIV deaths: -800
    Background deaths: -200
    Expected: 1,150,000
    Actual: 1,149,985
```

21.3 ART Mass Balance

Purpose: Ensure ART cascade accounting is correct - people don't get stuck or vanish.

Checks:

1. **Diagnosed population consistency**

```
diagnosed[t] = diagnosed[t-1] + new_diagnoses[t] - deaths_diagnosed[t]
```

2. **ART population consistency**

```
on_art[t] = on_art[t-1] + initiations[t] - dropouts[t] - deaths_on_art[t]
```

3. **Logical constraints**

```
on_art[t] ≤ diagnosed[t]  // Can't be on ART without diagnosis
diagnosed[t] ≤ hiv_positive[t]  // Can't be diagnosed without HIV
```

4. **Cascade flow conservation**

```
Undiagnosed → Diagnosed (via diagnosis)
Diagnosed, Not on ART → On ART (via initiation)
On ART → Not on ART (via dropout)
```

Tracked transitions:

- New diagnoses
- ART initiations
- ART dropouts
- Deaths at each stage (undiagnosed, diagnosed, on ART)

Example error:

```
ERROR: ART mass balance violation at timestep 25 (year 2005)

  Diagnosed balance:
    Previous diagnosed: 8,500
    New diagnoses: +500
    Diagnosed deaths: -200
    Expected: 8,800
    Actual: 8,750
    Discrepancy: -50

  ART balance:
    Previous on ART: 6,000
    Initiations: +400
    Dropouts: -150
    ART deaths: -100
    Expected: 6,150
    Actual: 6,200
    Discrepancy: +50
```

21.4 Enabling/Disabling Validation

Default: All validation enabled

To disable specific checks (not recommended):

```
"validation": {
  "transmission_sanity_checks": {"value": false},
  "mass_balance_checks": {"value": false},
  "art_balance_checks": {"value": false}
}
```

Recommendation: Keep all validation enabled for production runs. Only disable for debugging specific issues.

22. Conservation Checks

22.1 Population Initialization Checks

Run at initialization (not every timestep):

1. **Total population**

   ```
   allocated_total = sum(population tensor)
   expected_total = target_population
   tolerance = 0.001% (±10 people per million)
   ```

2. **Sexually active population**

   ```
   sexually_active = sum(population tensor)
   expected = sum(age_sex_distribution for ages ≥15) × target_population
   ```

3. **Childhood pool**

   ```
   childhood_total = sum(childhood tensor)
   expected = sum(age_sex_distribution for ages <15) × target_population
   ```

4. **Orientation totals**

   ```
   Female: sum across all Female dimensions
   Heterosexual_Male: sum across all Hetero_Male dimensions
   MSM: sum across all MSM dimensions

   Expected Female = 50% of total (approx, depends on sex ratio)
   Expected Hetero_Male = (1 − p_msm) × males
   ```

```
Expected MSM = p_msm × males
```

5. **Activity totals**

```
For each activity level:
  actual = sum(population[..., activity, ...])
  expected = total × activity_distribution[activity]
```

6. **No negatives**

```
For every cell in population tensor:
  value ≥ 0
```

22.2 Per-Timestep Checks (Runtime Validation)

Run every timestep:

1. **No negatives**

```
All population counts ≥ 0
All childhood counts ≥ 0
```

2. **No infinities/NaNs**

```
All values finite (not NaN, not Inf)
```

3. **Mass balance** (see Section 21.2)
4. **Transmission sanity** (see Section 21.1)
5. **ART balance** (see Section 21.3)

22.3 Error Reporting

Detailed, actionable error messages:

```
✓ Total population: 12,713,815 (target: 12,713,815, diff: 0)
✓ Sexually active: 11,500,000 (expected: 11,500,000, diff: 0)
✓ Childhood pool: 1,213,815 (expected: 1,213,815, diff: 0)
✓ Female orientation: 6,356,908 (expected: 6,356,908, diff: 0)
✓ Heterosexual_Male orientation: 5,721,217 (expected: 5,721,217, diff: 0)
✓ MSM orientation: 635,691 (expected: 635,691, diff: 0)
✓ Activity distribution validated
✓ No negative counts
```

Or:

```
❌ Conservation check failed: Total population mismatch
  Expected: 12,713,815
  Actual: 12,710,000
  Discrepancy: -3,815 (-0.03%)

  This suggests an allocation error in initialization.
  Check age_sex_distribution weights and target_population.
```

PART V: APPENDICES

Appendix A: Simulation Loop Order

V5 Timestep Sequence (executed once per timestep):

1. Population adjustments (if enabled)
2. **Time-scheduled rate lookup** (V5: ART, diagnosis, PrEP schedules)
3. Contacts & mixing
4. **Transmission** (with sanity checks)
5. Disease progression
6. Diagnosis (constant or time-scheduled)
7. ART transitions (constant or time-scheduled, with balance checks)
8. PrEP transitions (constant or time-scheduled)
9. Fertility/births (if enabled)
10. Mortality (background + HIV)
11. Sexual debut transitions (V4)
12. Aging (every 5 years)
13. **Mass balance validation** (V5)
14. Output recording

Order is critical and must not be changed.

Appendix B: Mathematical Notation

Rate Conversion:

```
p_step = 1 - (1 - p_annual)^(t_step / 1 year)
```

Transmission Probability:

```
P = β_s × (1 - ε_PrEP × c_PrEP) × m_condom × (1 - ε_ART × c_ART) × π
```

Where:

- β_s = stage-specific transmission probability
- ε_PrEP = PrEP efficacy
- c_PrEP = PrEP coverage in susceptible
- m_condom = condom modifier
- ε_ART = ART transmission reduction
- c_ART = ART coverage in infected
- π = prevalence in partner group

New Infections:

```
I_new ~ Binomial(n_contacts × π, P)
```

Fertility:

```
B = Σ_a (N_a^F × ASFR_a × t_step × m_year)
```

Where:

- N_a^F = women in age group a
- ASFR_a = age-specific fertility rate
- t_step = years per timestep
- m_year = year-specific multiplier

Mass Balance:

```
N(t+1) = N(t) + B(t) − D_HIV(t) − D_BG(t) + E(t)
```

Where:

- N = total population
- B = births
- D_HIV = HIV deaths
- D_BG = background deaths
- E = new entrants

Appendix C: File Structure

```
hiv_sim_v3_patch/
├── config.json                    # Base configuration
├── config_v5_test.json           # V5 test configuration
```

```
├── test_init_relative_weights.json  # Relative weights test
├── run_v3.py                    # Main simulation engine
├── run_scenarios.py             # V5 multi-scenario runner
├── engine_v3.py                 # Population state & initialization
├── demography.py                # V4 demography module
├── output_writer.py             # Output module
├── documentation/               # All documentation
│   ├── README_MODEL_GUIDE.md    # This document
│   ├── V5_PATCH_SUMMARY.md      # V5 features
│   ├── V5_QUICK_START.md        # V5 quick start
│   ├── VALIDATION_PATCHES.md    # Validation documentation
│   ├── POPULATION_INITIALIZATION_PATCH.md
│   ├── PATCH_OPEN_ENDED_AGE_BINS.md
│   ├── MODEL_REVIEW_REPORT.md
│   └── ...
└── outputs/                     # Generated outputs
    ├── baseline/
    │   ├── baseline_run_001.csv
    │   └── baseline_aggregated.csv
    └── ...
```

Appendix D: Configuration Hierarchy

```
config.json
├── meta (model info, version)
├── time (timestep system, start/end years)
├── population
│   ├── initialization_method (direct_counts vs relative_weights)
│   ├── target_population (for relative_weights)
│   ├── p_msm (for relative_weights)
│   ├── activity_distribution (for relative_weights)
│   ├── age_sex_distribution (for relative_weights)
│   ├── childhood_pool (auto, disabled, or manual)
│   ├── initial_distribution (for direct_counts)
│   └── hiv (initial prevalence)
├── demography (mode, mortality, entrants)
├── life_table (optional, age/sex-specific survival)
├── fertility (optional, ASFR-based births)
├── sexual_debut (V4, continuous debut modeling)
├── population_adjustments (optional, year-specific scaling)
├── groups
│   ├── age_groups (with open-ended bin support)
│   ├── orientations
│   └── sexual_activity_groups
├── mixing (sexual contact patterns)
├── disease (transmission, progression)
├── diagnosis (constant or time-scheduled)
├── treatment
│   ├── art_initiation_rate (if constant)
│   ├── art_dropout_rate (if constant)
│   └── art_schedule (V5, time-varying)
├── interventions (PrEP configs)
│   └── [PrEP_name]
│       ├── efficacy
│       ├── uptake_rate (if constant)
```

```
│       ├── dropout_rate (if constant)
│       └── prep_schedule (V5, time-varying)
├── policy (V5, baseline freeze & reuse)
├── scenarios (V5, multi-scenario framework)
├── output (file settings)
├── validation (enable/disable checks)
├── simulation_controls (n_runs, seed, deterministic mode)
└── notes (documentation)
```

Ordered by conceptual importance, not execution order.

Appendix E: Performance

Typical performance (modern CPU):

Metric	Value
40-year simulation (V3)	~0.5 seconds
40-year simulation (V5, extended age groups)	~0.8 seconds
100-seed ensemble	~50-80 seconds
Multi-scenario (3 scenarios × 10 seeds)	~150-240 seconds
Memory usage	~50-100 MB
Config validation	<50 ms

Optimizations:

- NumPy vectorized operations
- No loops over individuals
- Efficient state tensor
- Minimal Python overhead

Scaling:

- Linear with number of timesteps
- Linear with number of seeds
- Linear with number of scenarios
- Quadratic with number of age groups (but still fast)

Appendix F: Limitations

By Design (Intentional Simplifications):

1. Discrete timesteps (not continuous-time)
2. Group-based mixing (not explicit partnerships)
3. Homogeneous groups (individuals in same state identical)
4. No superinfection
5. No viral load stratification
6. No STI cofactors
7. No migration (can add via population adjustments)
8. No resistance (can model via separate ART states)

Technical Limitations:

1. Single population (no geographic stratification)
2. Fixed orientation (no transitions MSM ↔ Hetero)
3. Fixed HIV stages (cannot add CD4 stratification without code changes)

These are intentional simplifications appropriate for population-level cost-effectiveness modeling.

Appendix G: Validation Checklist

For external validators:

Configuration:

- [ ] Configuration loads without errors
- [ ] Mixing matrices sum to 1.0
- [ ] All probabilities in [0, 1]
- [ ] Age groups cover sexually active range
- [ ] Life tables extend to maximum age
- [ ] Fertility rates defined for reproductive ages

Initialization:

- [ ] Population initializes without errors
- [ ] Conservation checks pass (if using relative_weights)
- [ ] Total population matches target
- [ ] MSM proportion correct
- [ ] Activity distribution correct
- [ ] Childhood pool populated correctly (if enabled)

Simulation:

- [ ] No crashes during simulation
- [ ] Population never negative
- [ ] Mass balance preserved
- [ ] Transmission sanity checks pass

- [ ] ART balance checks pass
- [ ] Deterministic mode reproducible

Outputs:

- [ ] Output files created
- [ ] Terminal summary displays
- [ ] Multiple runs aggregate correctly
- [ ] Timeseries data makes sense
- [ ] Prevalence and incidence calculated correctly

V4 Features:

- [ ] Fertility produces births (if enabled)
- [ ] Life tables apply mortality (if enabled)
- [ ] Sexual debut transitions individuals (if enabled)
- [ ] Childhood pool ages correctly

V5 Features:

- [ ] Time-scheduled rates change over time
- [ ] Scenarios run independently
- [ ] Scenario outputs labeled correctly
- [ ] Baseline freeze works (if enabled)
- [ ] Validation patches catch errors

Appendix H: Version History

V1.0 (2023)

- Basic model with yearly timesteps
- Fixed dimensions
- Index-based configuration

V2.0 (2024)

- Dynamic configuration
- Flexible time system (days/months/years)
- Label-based groups
- PrEP interventions

V3.0 (2024)

- Robust outputs (CSV/Excel)
- Demography modules
- Life tables

- Comprehensive validation

V4.0 (2024-2025)

- Sexual debut system (continuous hazard)
- Childhood pool
- Flush-on-aging guardrails
- Enhanced fertility integration

V4.2

- Flush-on-aging implementation
- Conservation checks

V4.3

- Enhanced guardrails
- Validation improvements

V5.0 (2025) - **CURRENT**

- Time-scheduled interventions (ART, diagnosis, PrEP)
- Multi-scenario framework
- Policy comparisons (baseline freeze & reuse)
- Validation patches (transmission sanity, mass balance, ART balance)
- Population initialization methods (direct counts, relative weights)
- Extended age groups with open-ended bins
- Comprehensive conservation checks
- Census-based initialization

Quick Start Guide

For new users:

1. **Start with V3 basics** - Understand core model (Sections 1-11)
2. **Add V4 demography** - If modeling births/deaths (Sections 12-15)
3. **Explore V5 features** - For time-varying interventions (Sections 16-18)
4. **Customize** - Advanced initialization, extended ages (Sections 19-22)

For calibration:

1. Use **relative weights initialization** (Section 19.3)
2. Enable **time-scheduled ART/PrEP** (Section 16)
3. Use **life tables** for realistic mortality (Section 12.1)
4. Enable **all validation patches** (Section 21)
5. Run **multi-scenario comparisons** (Section 17)

For policy analysis:

1. Calibrate baseline 1980-2020
2. Enable **baseline freeze** at 2020 (Section 18)
3. Define **policy scenarios** 2020-2030 (Section 17)
4. Compare outcomes across scenarios

End of Model Guide

For detailed V5 usage examples, see `V5_QUICK_START.md`. For validation patch details, see `VALIDATION_PATCHES.md`. For population initialization details, see `POPULATION_INITIALIZATION_PATCH.md`. For open-ended age bin support, see `PATCH_OPEN_ENDED_AGE_BINS.md`.